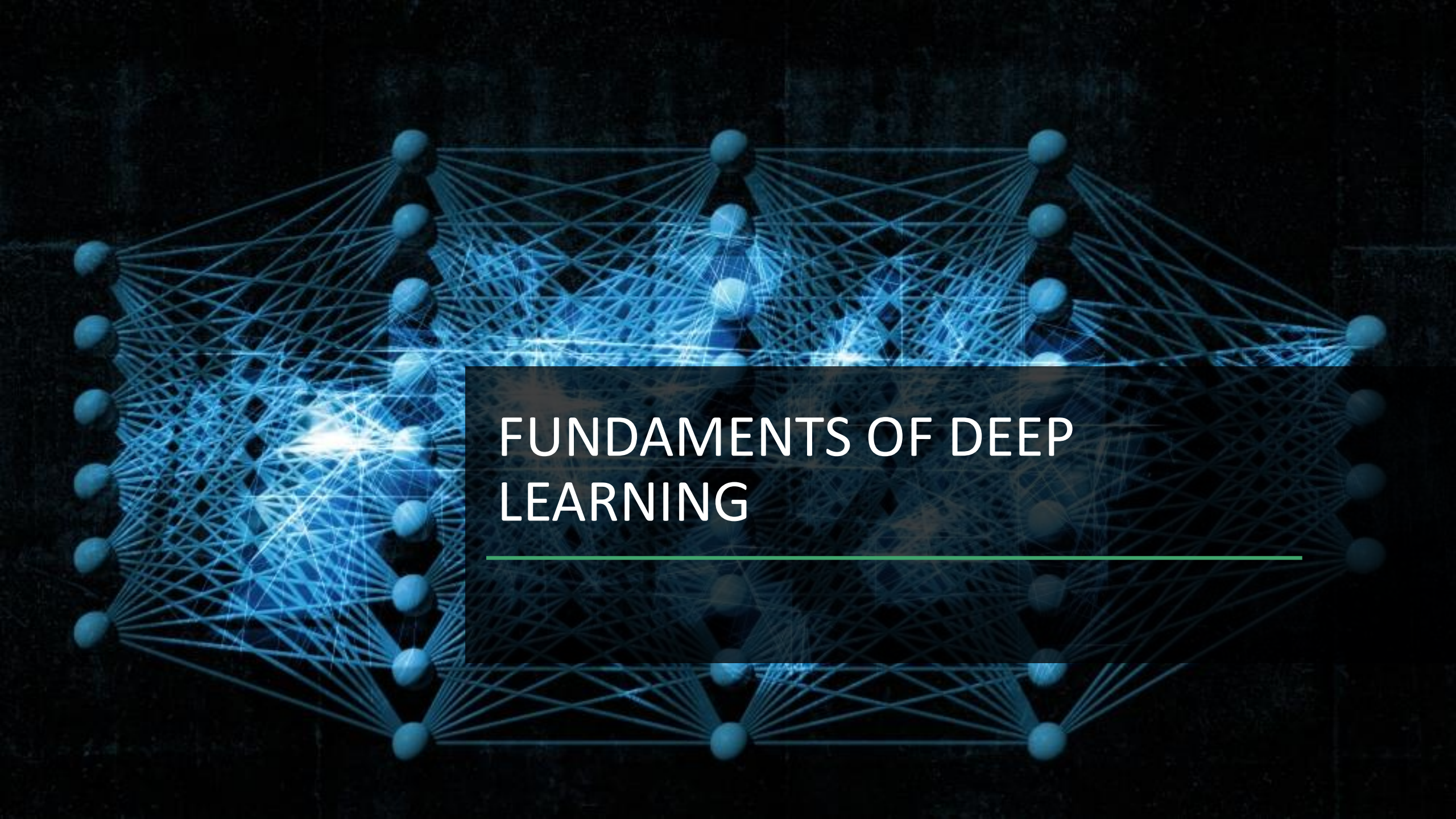




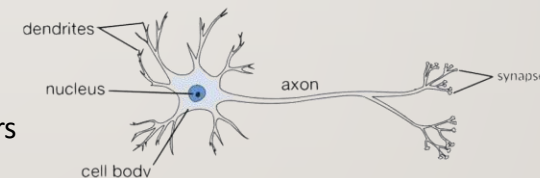
FUNDAMENTALS OF DEEP LEARNING

WHAT IS A DEEP LEARNING?

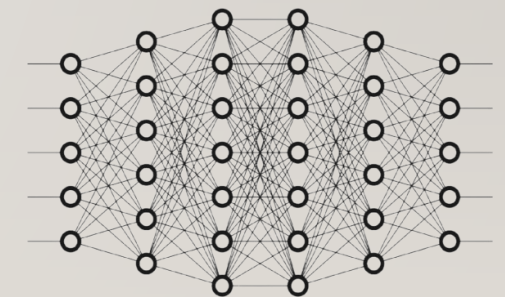
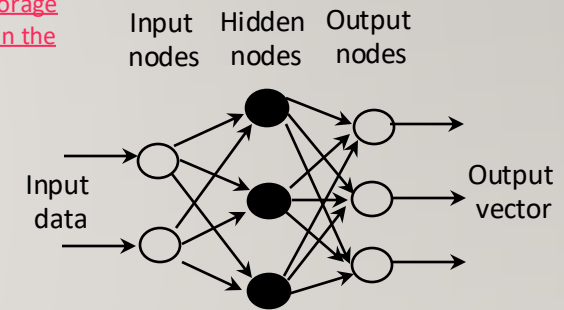
- Deep Learning refers to **neural networks**
 - consisting of nodes (*neurons*):
 - that process and transform input data producing an output
 - organized in *layers*:
 - **Input**: neurons getting the input data
 - **Hidden**: neurons in layers that are in the middle of the network
 - **Output**: providing the final output
 - connected through links (*synapses*):
 - that pass the information between neurons in different layers
 - Having.. 3 or more hidden-layers
- Deep Neural Network can have very different structures and **architectures!**
 - Feed-Forward, Recurrent, Graph, Convolutional, etc..
 - Importance of **model engineering**
- But everything starts from a **single neuron**
 - And **the idea of the perceptron.. Back in time**



Biological Neuron



The Perceptron:
A Probabilistic Model
for Information Storage
and Organization in the
Brain
[1958]



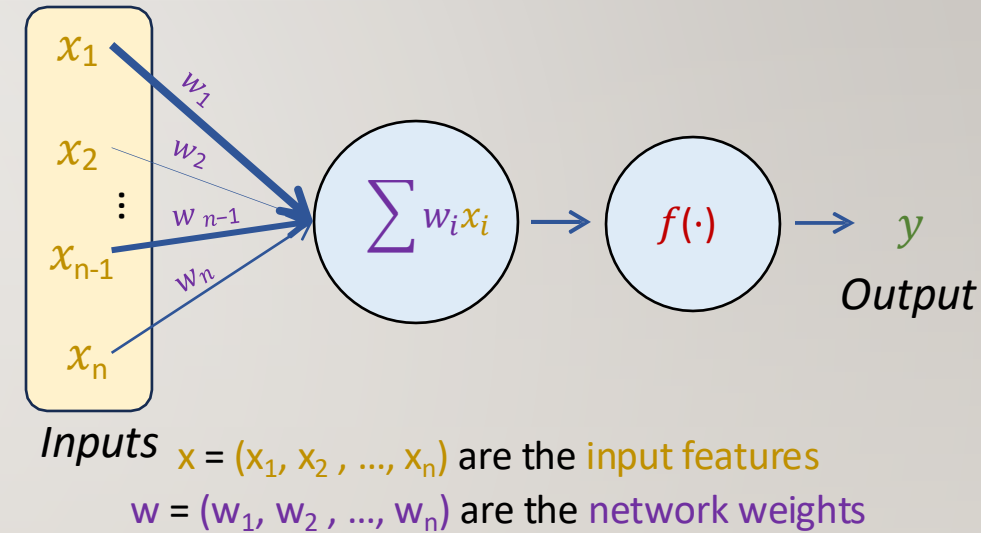
Deep Neural Network
4 hidden-layers!

INTRODUCTION TO THE PERCEPTRON

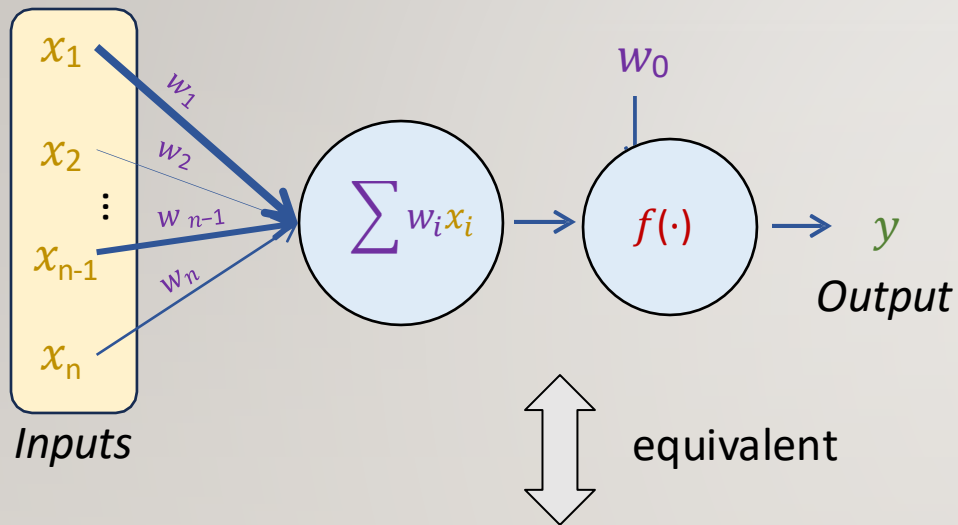
THE PERCEPTRON

- The perceptron is the simplest unit of neural networks
- It takes an input with *multiple features*, and:
 - It weights each input *feature* with a given *weight*
 - It produces a weighted sum of the inputs, and
 - It applies a *function* to the weighted sum and produces an *output*

$$y = f(w_1x_1 + w_2x_2 + \dots + w_nx_n)$$



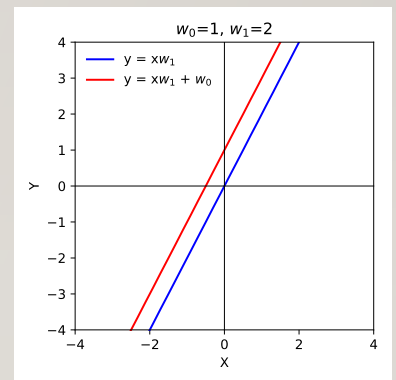
THE PERCEPTRON – MORE DETAILS



$$y = f(\sum_{i=0}^n w_i x_i) = f(w^T x) \text{ and } x_0 = 1^*$$

x_0 is 1 by definition. It represents the feature that multiply the bias w_0

- $x = (x_1, x_2, \dots, x_n)$ is the input sample – the features
- y represents the output of the perceptron
- $f(\cdot)$ represents a non-linear “activation” function
- w_i are **weights**, while w_0 is the **bias**
 - w_0 is introduced to **shift** the input to the activation function to represent any possible linear function
- All w are “learned” from the data



Note:

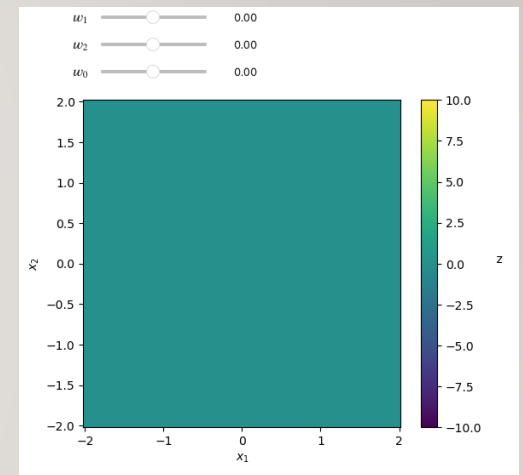
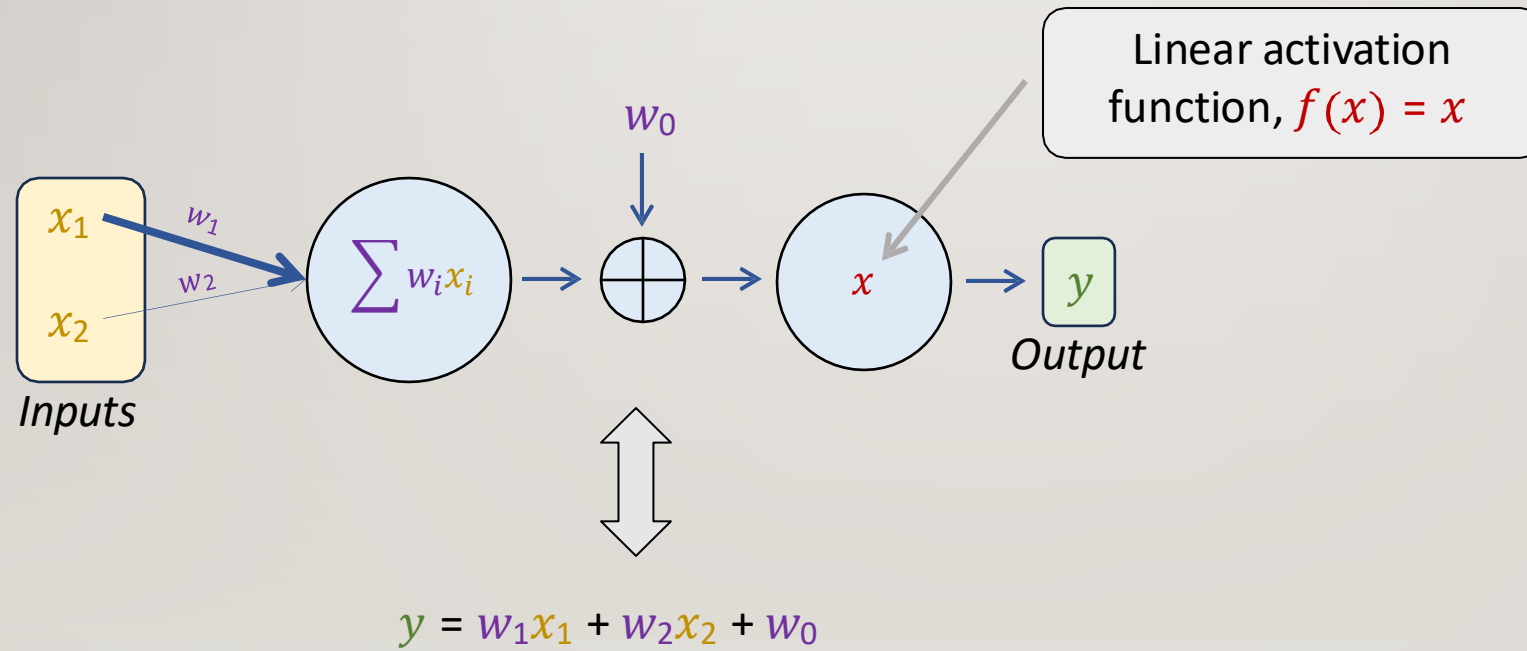
With the exception of $f(\cdot)$, this looks like the classic *linear regression*

And if $f(\cdot) = \sigma(\cdot)$ (sigmoid function), this looks like the (just as classic) *logistic regression*

THE PERCEPTRON - 2D SCENARIO, LINEAR ACTIVATION

For a 2D scenario*, with $\mathbf{x}=(x_1, x_2)$ and $\mathbf{w}=(w_0, w_1, w_2)$:

*Given the height and weight of a class of students, determine gender



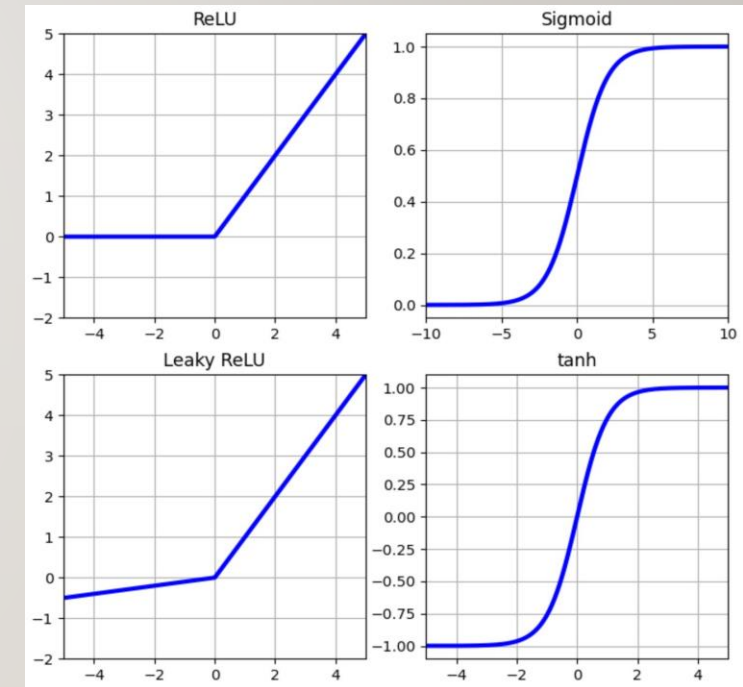
The perceptron can be used to represent a family of functions,

$$y = w_1 x_1 + w_2 x_2 + w_0$$

Various values of w_0, w_1, w_2 define the different functions that can be learned by the perceptron.

ACTIVATION FUNCTIONS

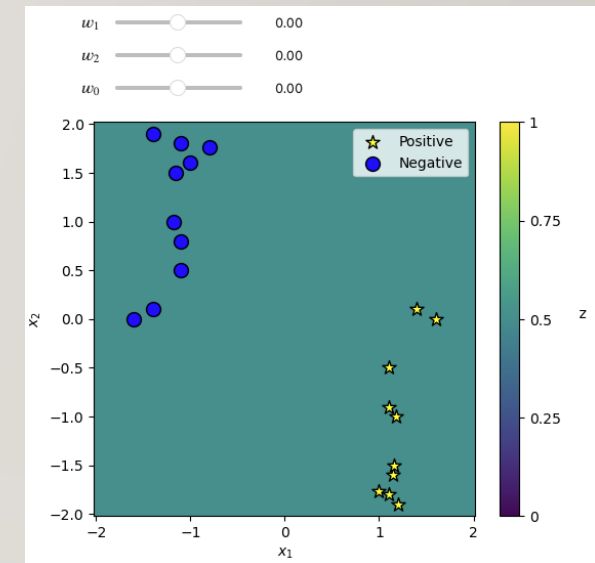
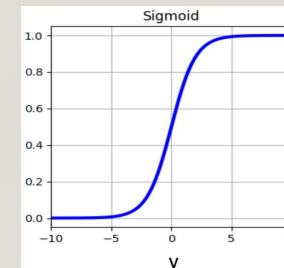
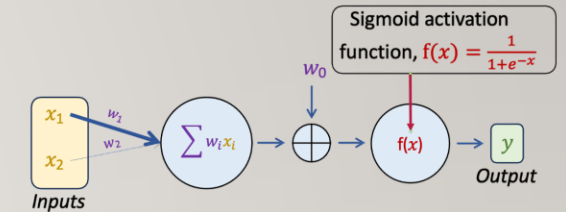
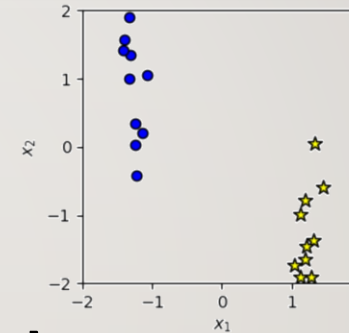
- Activation functions are used for different reasons:
 1. Enforce properties on perceptron's output
 - E.g., sigmoid → binds output to $[0, 1]$ range
 2. Introduce non-linearities in the model
 3. Decide when the perceptron is active
 - E.g., ReLU → if the input of the activation function is 0 the neuron is not active
- Other optimization reasons (faster convergence, sparsity, ...)
- Commonly adopted functions:
 - ReLU, Sigmoid, Leaky ReLU, Tanh, Softmax, Linear, GeLU, ...
 - Fell free to explore an extended list [here](#)
 - Some of them may have problem with **very** Deep Neural Network
 - E.g., gradient vanishing problem



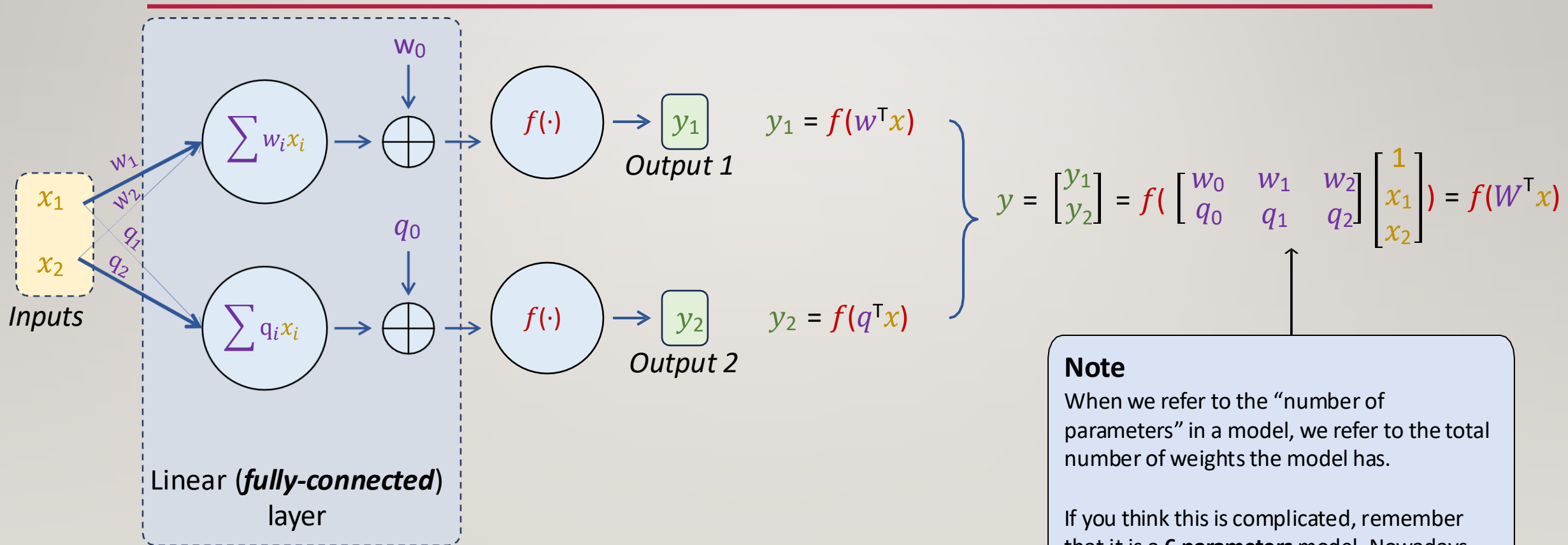
EXAMPLE OF ACTIVATION: SIGMOID FUNCTION

- Binary classification problem
 - Separate positive (★) and negative (●) samples
- For a point $x \in \mathbb{R}^2$, the perceptron can predict $p(\star | x)$
 - For the binary case, this implies $p(\bullet | x) = 1 - p(\star | x)$
- To get a valid probability, we must enforce $p(\star | x) \in [0, 1]$
 - We already have $p(\bullet | x) + p(\star | x) = 1$ by construction
- The Sigmoid activation function maps **any** value in $\mathbb{R}$ to the range $[0, 1]$
 - i.e., the perceptron's output (in $\mathbb{R}$) is squashed to $[0, 1]$

$$\text{Sigmoid} = \sigma(y) = \frac{1}{1 + e^{-y}}$$



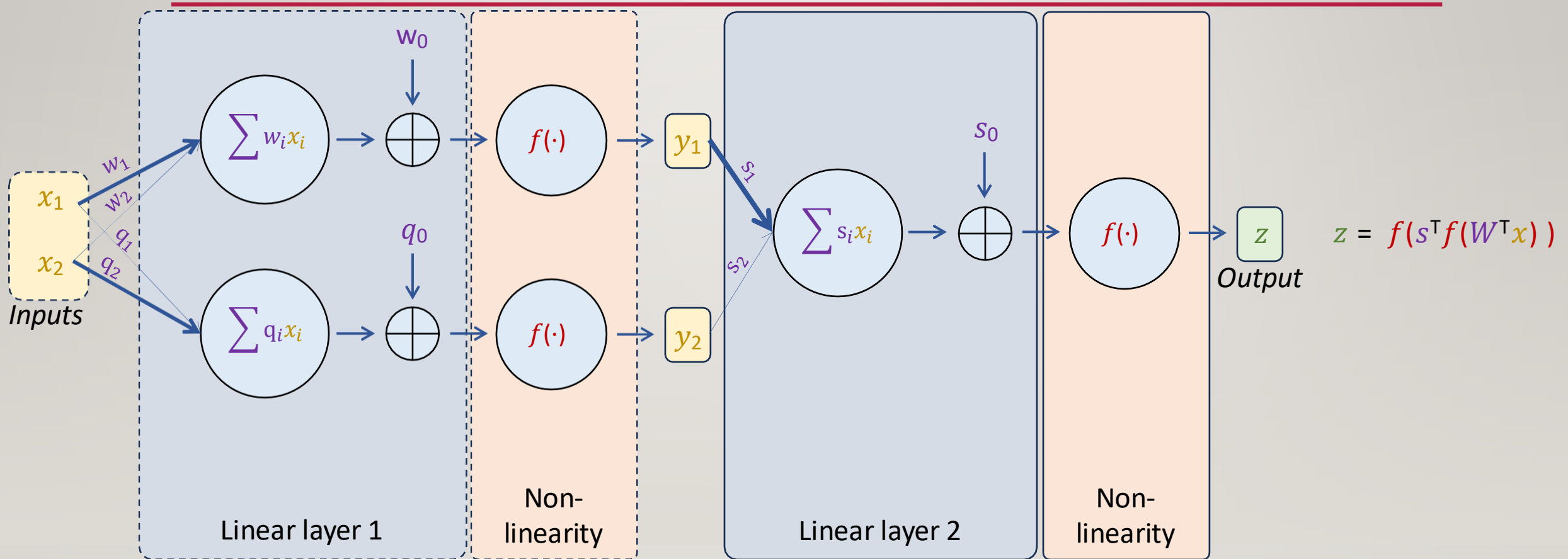
TOWARDS BIGGER BRAINS: STACKING PERCEPTRONS...



When we refer to the “number of parameters” in a model, we refer to the total number of weights the model has.

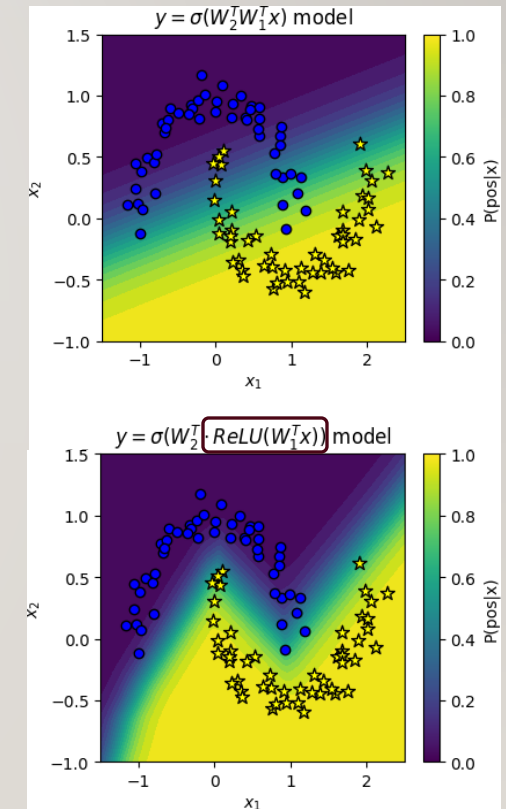
If you think this is complicated, remember that it is a **6 parameters** model. Nowadays, researchers sometimes call “**small**” models with **7B parameters** (e.g., [Mistral 7B](#))

TOWARD BIGGER BRAINS: ...AND ADDING LAYERS!



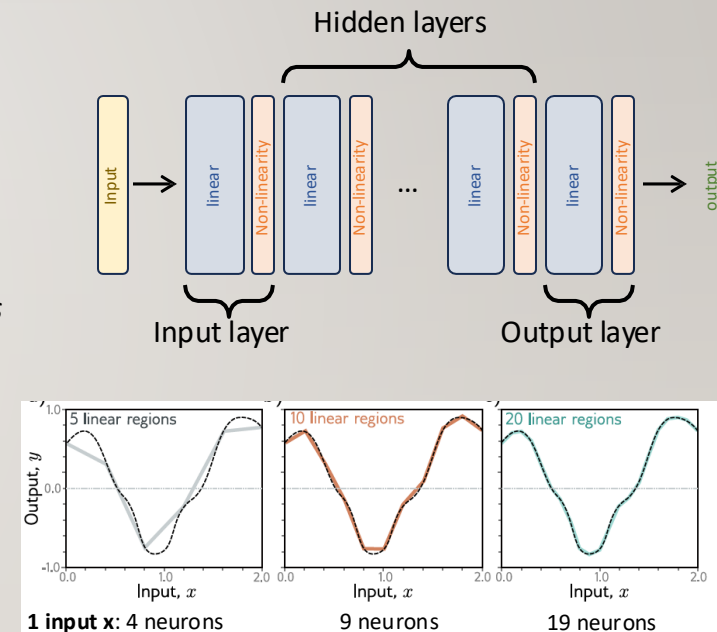
INTRODUCE **NON-LINEARITIES** IN THE MODEL

- if $f(x) = x$ (i.e., no non-linearity is added), we get $z = s^T W^T x$
- This implies:
 1. We could have used $W' = Ws$ and get the same output
 2. We wouldn't have needed a second layer!
 3. But our model is still linear
- So, we use the ReLU **non-linear** activation functions to model more complex functions



MULTI-LAYER PERCEPTRON MODELS

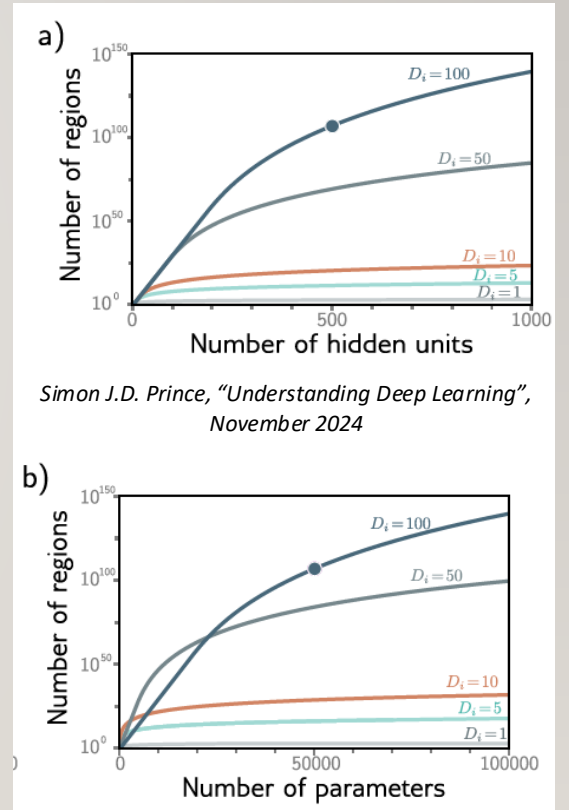
- We can stack additional **layers**
 - separated by **non-linearities** (activation functions) to prevent collapses
 - Collapse as **without** non-linearities multiple layers behave like a **single linear transformation**
- *Universal Approximation Theorem* tells us that we can approximate “**any**” function with a **single hidden layer**
 - “For any continuous function g defined on a compact subset of $\mathbb{R}^m$ and for any $\epsilon > 0$, there exists a feedforward neural network with a **single hidden layer** and a finite number of neurons that can approximate g to within an arbitrary degree of accuracy ϵ ”*
- Here with a **linear model** with 19 linear neurons we can approximate the curve
 - This curve is described by 20 linear regions
- A single-layer MLP works ... but no information on the number of neurons, or the weights’ values!
- Deeper, narrower networks are generally used
 - More layers creates more abstract data representation improving generalization, as deeper layer create more abstract data representation



Simon J.D. Prince, “Understanding Deep Learning”, November 2024

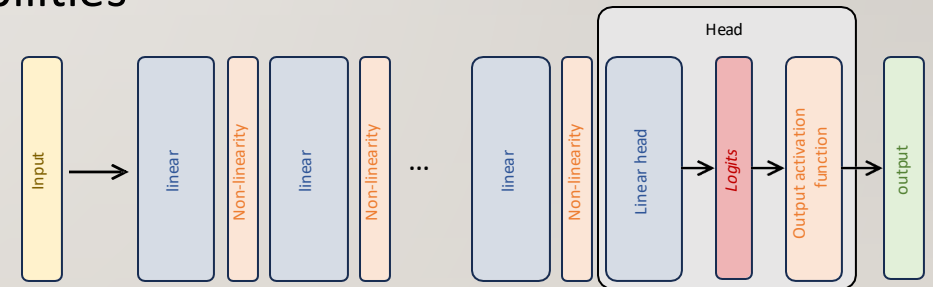
NETWORK COMPLEXITY

- Number of Linear regions with **1 linear hidden layer**
- Same example as before increasing D_i = **number of features**
- **a:** number of regions w.r.t. number of hidden neurons and features
 - The number of regions increases rapidly with the number of features
 - E.g., with 500 neurons and input size $D_i = 100$ features > 10.107 regions
- **b:** Same plot with respect to the number of neural network parameters to tune
 - Solid circle is the model as in figure a) (500 neurons, 100 features)
 - This network has **51,001** parameters!
 - A lot? Not, really! It is **very small** by modern standards! **Recall 7B Mitral!**



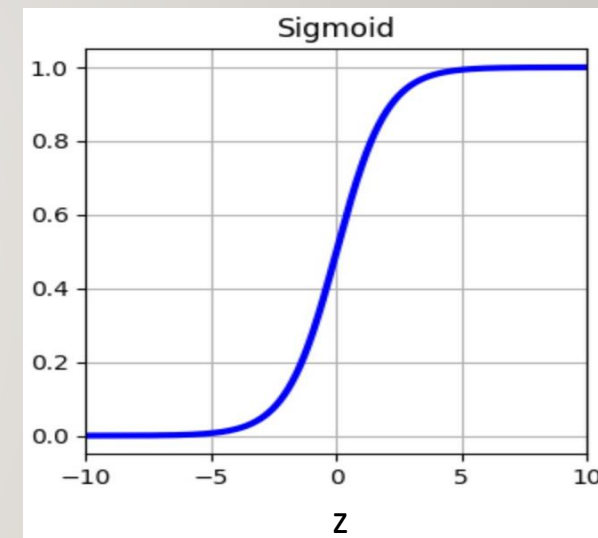
ACTIVATION FUNCTIONS FOR CLASSIFICATION MODELS

- As argued, activation functions can be used to enforce properties on the model's output
- In classification problems, the output before the final activation is treated as **unnormalized probabilities (logits)**
- We still need a step to convert logits into valid probabilities
 - i.e., all probabilities should sum to 1, and be in $[0, 1]$



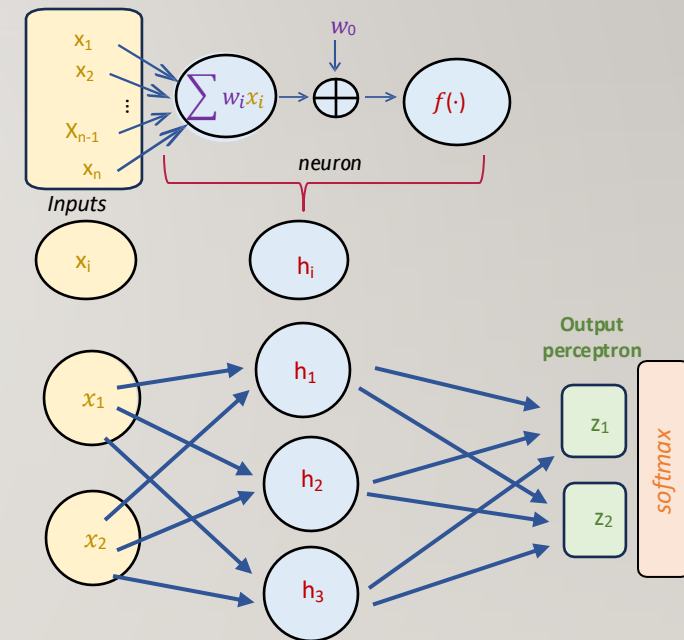
BINARY CLASSIFICATION

- The model predicts the probability of a single class for point x
 - As a convention, the positive one $P(pos|x)$
- The model produces a logit $z = model(x)$
- We use the **sigmoid function** on the output logit z
 - $\sigma(z) = \frac{1}{1+e^{-z}}$
 - This guarantees $P pos x \in [0, 1]$ ✓
- We work out the probability of the negative class
 - $P(neg|x) = 1 - P(pos|x)$
- We can easily show that $P(neg|x) \in [0,1]$ ✓
- By construction, $P(pos|x) + P(neg|x) = 1$ ✓

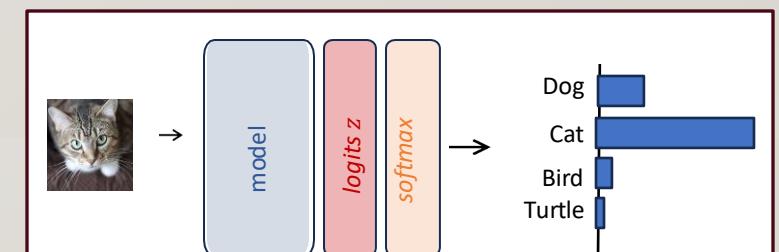


MULTI-CLASS CLASSIFICATION

- The output class y_i is one of many $c_1, c_2, \dots, c_n$
- The model **produces n logits** for a point x
 - i.e., the last layer will have n perceptron
 - $z = (z_1, z_2, \dots, z_n) = \text{model}(x)$
- We need to obtain, **from the logits**, valid **probabilities**
 - $P(c_1|x), P(c_2|x), \dots, P(c_n|x)$
- The **softmax** function is applied
 - $$\sum_i P(c_i|x) = \frac{e^{z_j}}{\sum_j e^{z_j}}$$
 - The softmax as activation function uses the **output of all output neurons together** to extract the classes probability!
- It can be easily shown that:
 - $P(c_i|x) \in [0,1]$ ✓
 - $\sum_i P(c_i|x) = 1$ ✓



20 parameters: 15 weights + 5 offsets



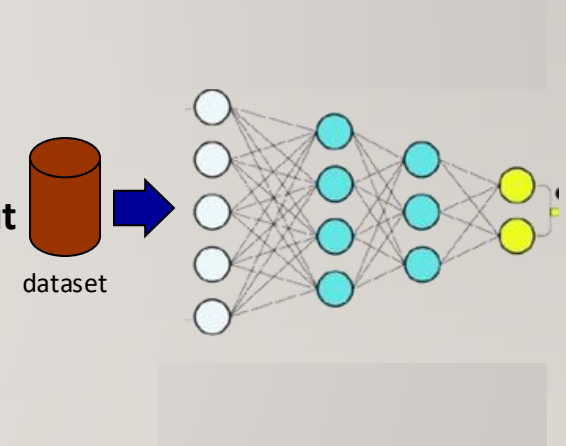
ACTIVATION FUNCTIONS FOR REGRESSION MODELS

- In regression, models generally predict real numbers
- Typically, there is no need to enforce properties
- The activation function in the **output layer** can be the identity function
 - $f(x) = x$
 - Generally, the only situation where it makes sense to use it!

HOW TO DEFINE THE NETWORK PARAMETERS?

DEFINING WEIGHTS (PARAMETERS)

- So far, we assumed all weights and biases (let's call them ϕ) to be known
 - But we still need to figure out how we find them!
- We iterate multiple time over the **training set** to tune the model weights
 - We input the training set multiple times
 - Each iteration is called an **epoch** – we send the **data forward** in the network
 - The weights are adjusted based on the **back-propagation of the errors**
 - The error is computed based on the **difference** between the **supposed output** and the **real output**
- We can adjust the weights considering all data together
 - Remember we may use huge amount of data..
 - Does it fit in memory?
 - How long does it take before computing the back-propagation error?
- Dividing the data in **mini-batch** - A subset of data with **random samples**
 - They are typically created **without replacement**
 - Improve model generalization.. And scalability



DEFINING WEIGHTS (PARAMETERS) - CONTINUE

- We define a **learning rate** α to control how much the model's weights are updated during training
 - It determines the **step size** in the **direction of the gradient** during optimization
- Then we pick a function (objective, or loss), $\mathcal{L}(\phi)$, that we want to **minimize**
 - E.g., in Linear Regression we minimize the Mean Squared Error
 - $\mathcal{L}(\phi) = MSE(\phi) = \frac{1}{N} \sum (y_i - \phi^T x_i)^2$
- Then, we compute ϕ that minimizes it

Note

$\mathcal{L}$ also depends on the training points x_i, y_i , so we should refer to it as $\mathcal{L}(\phi, X, y)$

However, the training set X, y is generally fixed. Thus, we only have control over ϕ , so we use the notation $\mathcal{L}(\phi)$

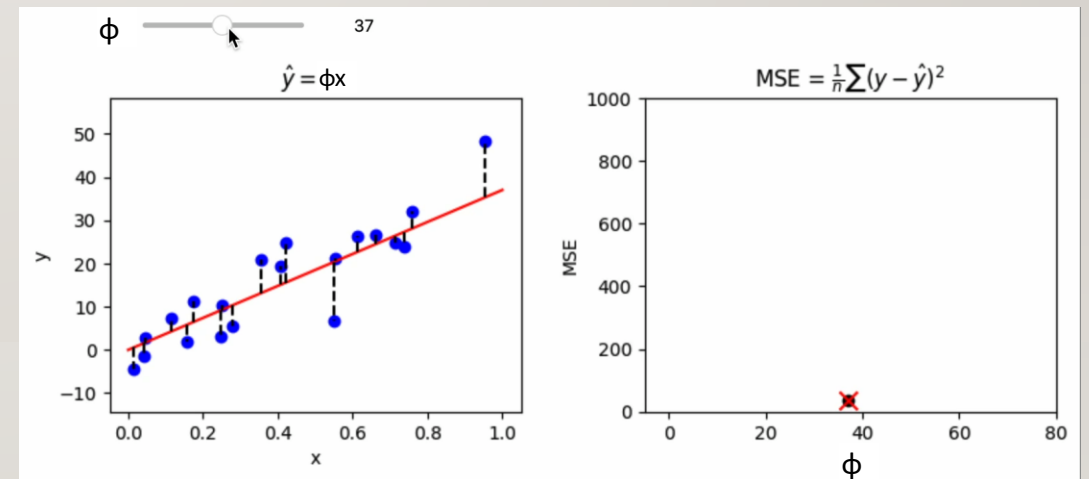
LINEAR REGRESSION

- For simple models, we can find the optimal weights in closed form
 - I.e., We can resolve analytically
 - $\frac{\partial \mathcal{L}(\phi)}{\partial \phi} = \frac{\partial \text{MSE}(\phi)}{\partial \phi} = 0$
 - Quadratic in ϕ , can be solved easily!
- Or, we can evaluate the Loss function for a different values of ϕ 's, and find the “best” one

Note

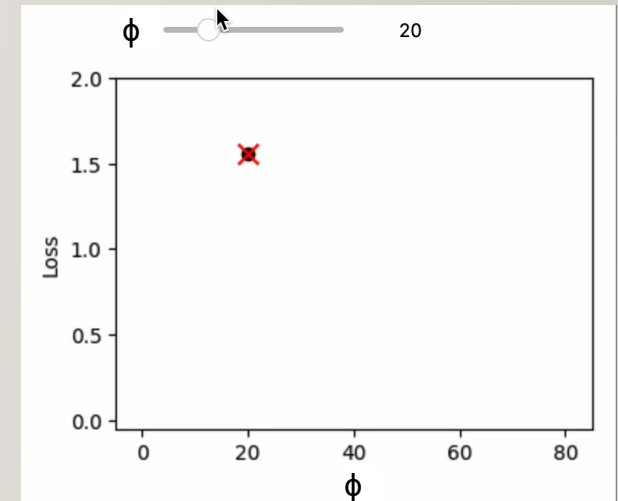
For linear regression, we don't try a different values of ϕ since we can easily find the best value in closed form..

However, this provides the intuition for what we will do next with more complex loss functions/models.



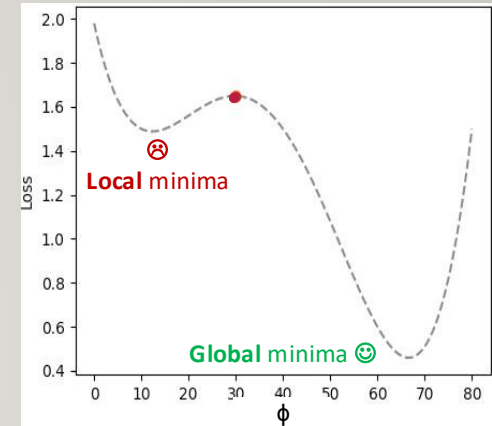
MORE COMPLEX LOSSES/MODELS

- For *more complex* loss functions/models, we may not be able to solve the problem in closed form
 - But we can evaluate $\mathcal{L}(\phi)$ for different values of ϕ
- We can iteratively update ϕ to reach a local minimum:
 - We start from a random value ϕ , then
 - we “**move around**” according to “**some policy**” using an optimization algorithms



WE “MOVE AROUND” ACCORDING TO AN OPTIMIZATION ALGORITHM

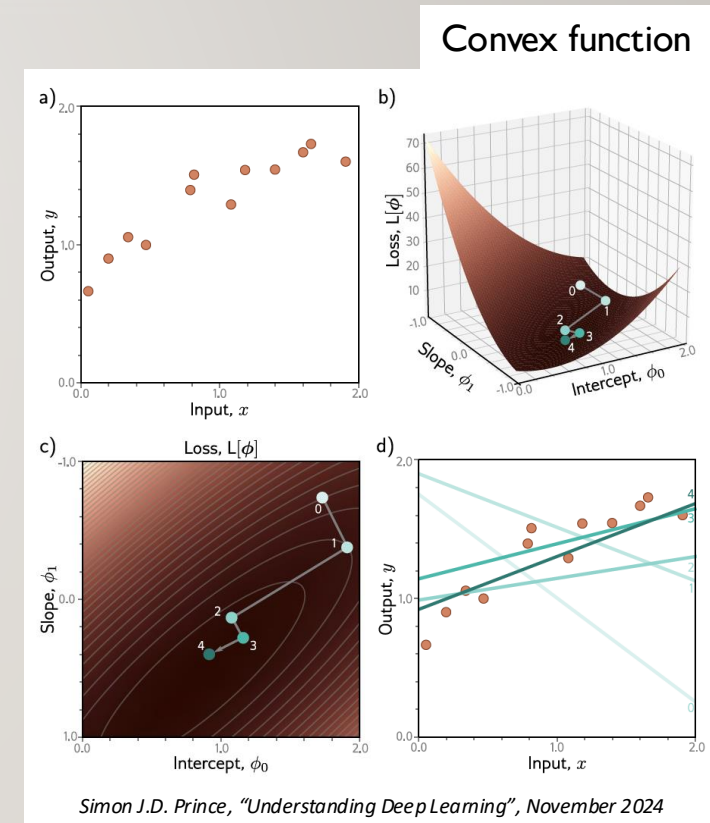
- Use **all training data together** to compute the Loss
- Move around = update ϕ incrementally, based on its current value
 - The new value of ϕ at any step depends on the previous step's value
 - $\phi_{t+1} := \phi_t + \text{update}$
- Some policy = we take a small step in the direction where the Loss function decreases locally
 - I.e., in the **opposite** direction of the **gradient**
 - $\phi_{t+1} := \phi_t - \alpha \nabla_{\phi} \mathcal{L}(\phi_t)$
 - for 1-dimensional ϕ , we have $\phi_{t+1} := \phi_t - \alpha \frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}$
 - α : **the learning rate** controls the “size” of the step
 - Typically small: take **small step per time**
- Gradient Descent!
 - It works well with convex functions



GD WITH A SIMPLE LINEAR REGRESSION A TOY EXAMPLE:

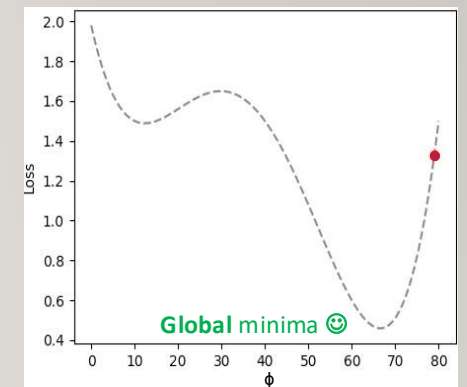
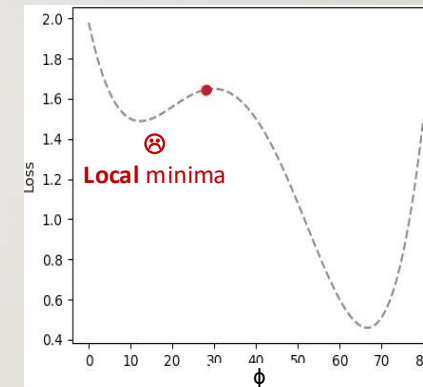
$$Y = X\Phi_1 + \Phi_0$$

- **a:** Training set of $I = 12$ input/output pairs $\{x_i, y_i\}$.
- **b: Loss function** showing iterations of gradient descent
 - Start at point 0 (random initialization) and move in the steepest downhill direction
 - Until we can improve no further to arrive at point 1
 - Repeat this procedure form several epochs
 - Until a stopping criteria
 - E.g., we find a local minima, we reached a maximum number of epochs
- **c:** heatmap of the 3D plot
 - the line is the loss path
 - The brightness the colour the higher the loss
- After only four iterations, we are already close to the minimum!
- **d:** How model ($y = x\phi_1 + \phi_0$) describe the data
 - Lighter colour represent the first steps
 - Dark green is the last (and best approximation)

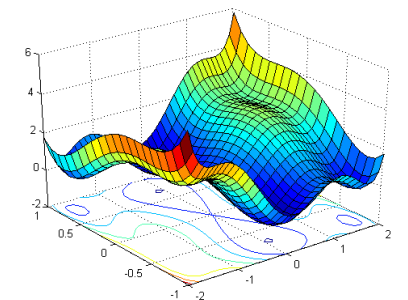


GD IS THE SOLUTION?

- GD works but it is sensitive to weight initialization
 - Different initializations can lead to different solutions!
 - It does not work well with **no-convex** loss function
 - GD can get stuck in local minima
 - GD **use all data** to decide the direction
- Various solutions to help prevent local minima
 - Stochastic Gradient Descent
 - Add Momentum
 - ADAM
 - ..



Non-convex function

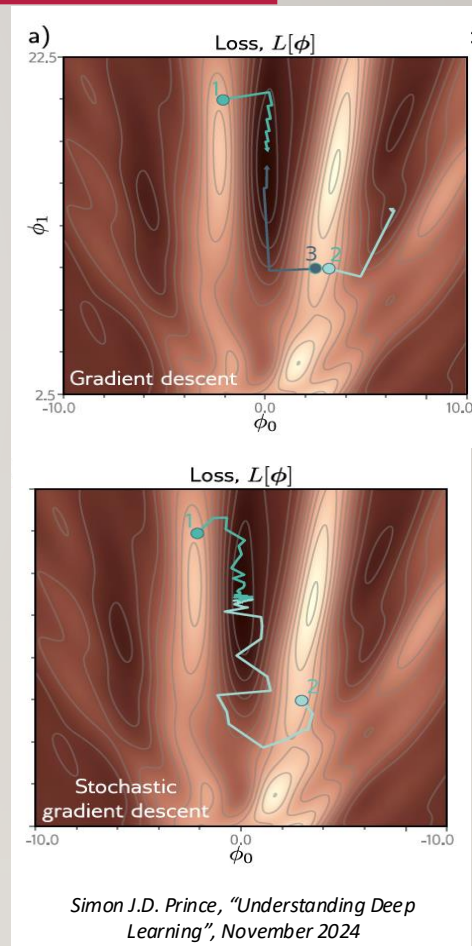


Note

In convex function different initializations will always lead to the global minimum! 😊
However, the Loss curve is typically a non-convex problem 😞

STOCHASTIC GRADIENT DESCENT

- **GD** totally depends on weights initialization, and use points in the dataset to compute the Loss
- Stochastic gradient descent (**SGD**) adds **stochasticity** computing the gradient at each step using only a **random mini-batch of data** per time
 - for 1-dimensional ϕ , we have $\phi_{t+1} := \phi_t - \alpha \left[\sum_{i \in B_t} \frac{\partial \mathcal{L}_i(\phi_t)}{\partial \phi} \right]$
 - B_t are the points in the batch of data
 - Random mini-batch is extracted **without replacement** – **each minibatch** is used **only once per epoch**
 - The solution still moves downhill **on average**
 - But at any given iteration the direction chosen is not necessarily in the steepest downhill direction
 - It might not be downhill at all!
- The SGD algorithm has the possibility of moving **temporarily** uphill and hence jumping as well
 - Can jump from a "valley" (local minima) of the loss function to another
- **GD**: With 3 different initializations
 - If random initialization is 1 or 3 the global minima is found! 😊 2 no! 😞
- **SGD**: Challenge SGD with the same initialization 1 and 2
 - Both lead to the global minima! 😊
 - Faster convergence



STOCHASTIC PROPERTIES 1/2

1. Although it adds noise to **the trajectory**, it may improve the overall Loss after a whole epoch
 - The **updates** tend to be **sensible** even if they are not optimal
2. Mini-batch example are taken **without replacement** and iterates through the dataset
 - All the training examples contribute **equally**
3. Faster than GD since it updates weights without waiting to read the full dataset
4. It is **less computationally expensive** to compute the gradient from just a subset of the training data
 - The mini-batch easily fits in the main memory as well
5. It can (in principle) escape local minima
6. It reduces the chances of getting stuck near saddle points
 - At least some of the possible batches will have a significant gradient at any point on the loss function
7. Evidence shows that SGD finds parameters for neural networks that cause them to generalize well to new data in practice!

STOCHASTIC PROPERTIES 2/2

- SGD does not necessarily “converge” in the traditional sense
 - The idea is that if we are close to the global minimum, all the data points will be well described by the model
 - ➔ Consequently, the gradient will be small, whichever batch is chosen, and the parameters will not change much the loss
- SGD is often applied with a **learning rate schedule**: α starts at a **high** value and is **decreased** by a constant factor **every N epochs**
 - In the **early stages** of training: the algorithm quickly explores the parameter space
 - Can jump from valley to valley to find a sensible region – move more and faster
 - In **later stages**: the loss is close to the minima
 - Fine-tuning the weights, so α decreases to make smaller changes

ADDING MOMENTUM

Modify the gradient at each micro-batch **may** introduce a lot of variability

→ Leading to **random oscillations**

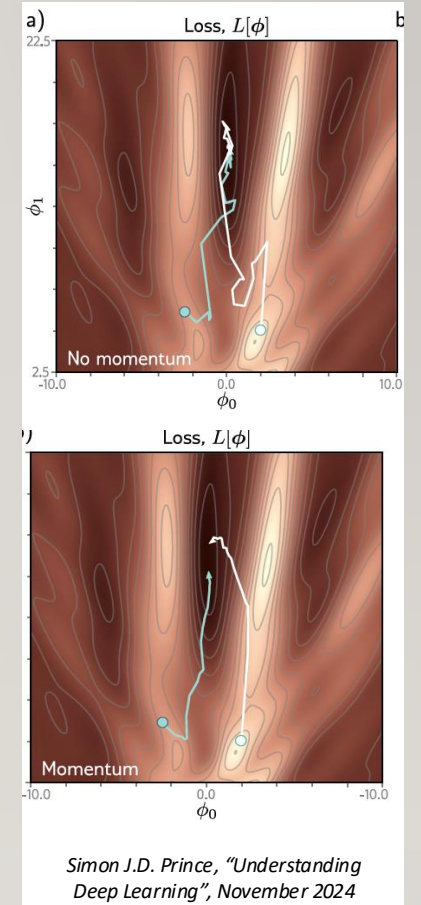
What about remember how the gradients where previously updated?

- Modify the Stochastic Gradient Descent adding a **momentum term** to remember the past
- Parameters updates is based on a a weighted combination of
 - the **gradient** computed from the **current batch**
 - the **direction** moved in the **previous step** – *the momentum*

1.
$$\mathbf{m}_{t+1} := \beta \mathbf{m}_t + (1-\beta) \sum_{i \in B_t} \frac{\partial \mathcal{L}_i(\phi_t)}{\partial \phi}$$
 - $\mathbf{m}_t$ is the momentum at iteration t
 - Driving the update direction
 - β describe how the gradient is smoother over time

2.
$$\phi_{t+1} := \phi_t - \alpha \mathbf{m}_{t+1}$$

The overall effect is a **smoother trajectory** and reduced oscillatory behaviour in valleys



FIXED LEARNING RATE: SOME LIMITATIONS

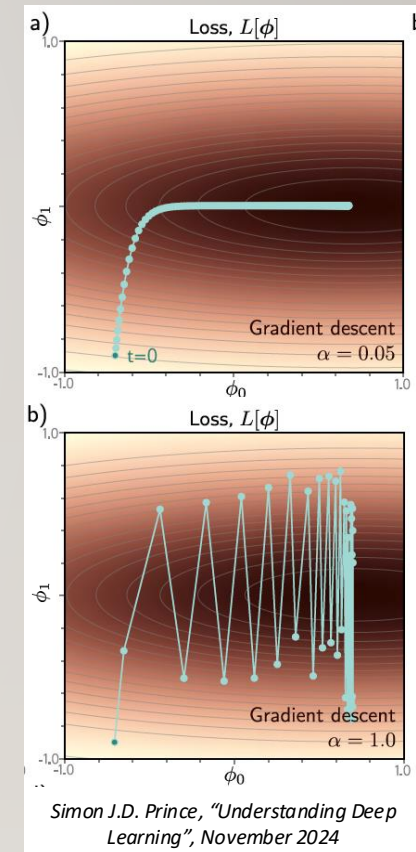
- Gradient descent with a **fixed learning** rate α makes:
 - Large adjustments to parameters associated with large gradients
 - That we may have already explored a lot
 - Small adjustments to parameters associated with small gradients
 - That we should explore more

a: small learning rate α

- Quickly converging on ϕ_1
 - It has the greatest impact on the loss - steeper direction
- Slowly on ϕ_2

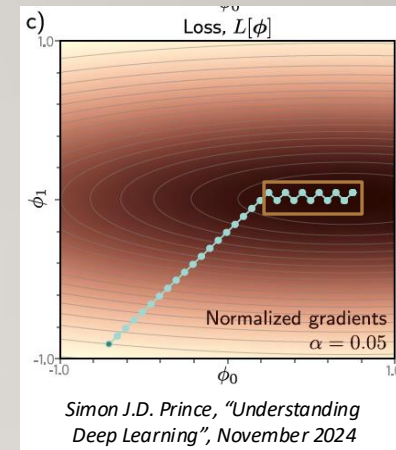
b: large learning rate α

- Very unstable! Both ϕ change a lot



GRADIENT NORMALIZATION

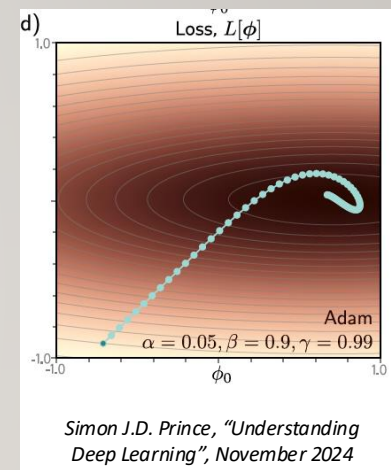
- First solution **normalize the gradients**
 - In this case using all data
 1. Momentum: $m_{t+1} := \frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}$
 2. Second-momentum as squared gradient $v_{t+1} := \left(\frac{\partial \mathcal{L}(\phi_t)}{\partial \phi} \right)^2$
 3. Update $\phi_{t+1} := \phi_t - \alpha \frac{m_{t+1}}{\sqrt{v_{t+1}} + \epsilon}$
 - ϵ is a small constant to avoid division by 0
 - **Gradient normalization**
 - **The result give us the sign for each coordinate direction moving of a fixed distance α**
- All directions have equal importance due to the normalization that
 - Reduces the importance and change of the directions with high gradient
 - Keep the importance of the other directions



- **NOTICE:** This approach may oscillate around the local minima

ADAPTIVE MOMENTUM ESTIMATION (ADAM)

- Adam, **adds the momentum to both the estimate of the gradient and the squared gradient**
 - In this case using all data, **typically computed using mini-batches**
 - 1. Momentum $m_{t+1} := \beta m_t + (1-\beta) \frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}$
 - 2. Second-momentum as squared gradient $v_{t+1} := \gamma v_t + (1-\gamma) \left(\frac{\partial \mathcal{L}(\phi_t)}{\partial \phi} \right)^2$
 - β and γ are the momentum coefficients between $[0,1]$
 - ➔ Initially m_{t+1} and v_{t+1} are effectively 0 with an unrealistic small estimation
 - 4. Bias-Corrected First Moment Estimate: $\hat{m}_{t+1} = \frac{m_{t+1}}{1-\beta^{t+1}}$
 - 5. Bias-Corrected Second Moment Estimate: $\hat{v}_{t+1} = \frac{v_{t+1}}{1-\gamma^{t+1}}$
 - The exponent over time **decay the correction** exponentially towards zero
 - It is significant in the early steps but negligible later
 - 6. Update $\phi_{t+1} := \phi_t - \alpha \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \epsilon}}$
- Improvement: faster convergence and stability**, especially for sparse gradients

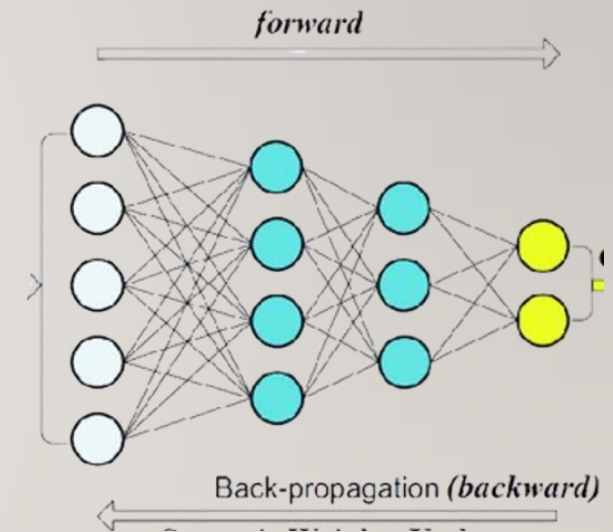


SUMMARY

- The optimization algorithm, batch size, learning rate, and momentum coefficients are all considered **hyperparameters** to choose during training
- They directly affects the final model performance but are distinct from the model parameters and architecture
- Choosing these can be more art than science
 - Typically we train many models with different hyperparameters and choose the best one
 - This is known as hyperparameter search

BACKPROPAGATION

- So far, we assumed we were able to compute $\nabla_{\phi} \mathcal{L}(\phi)$
- However, any loss/model combination would need a different gradient computation!
- We can use backpropagation to compute the gradient of the loss w.r.t. any weight!
 - Backpropagation is just a fancy word for “using the chain rule”

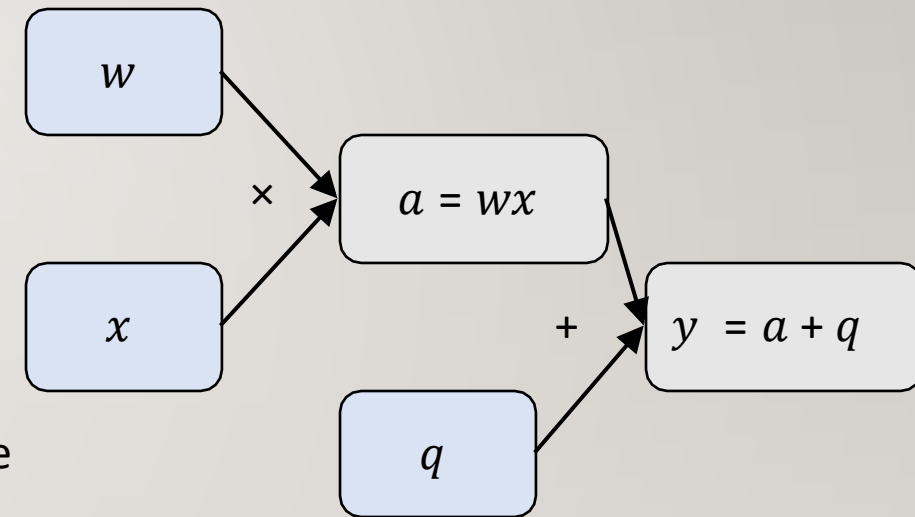


USING THE CHAIN RULE

- We use the chain rule from calculus, $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$
 - Sometimes known as $f(g(x))' = f'(g(x))g'(x)$
- And apply it **from** the end of the **computational graph**, backwards
 - (hence the name, backpropagation)

COMPUTATIONAL GRAPH

- A computational graph is a directed graph
 - Each node corresponds to an operation
 - Each edge represents the flow of data between nodes
- For instance, we may want to compute $y = wx + q$
 - We start from three variables, w , x and y
 - The computational graph performs one operation at a time
 - First, compute the intermediate variable $a = wx$
 - Then, compute the output variable $z = a + y = wx + y$

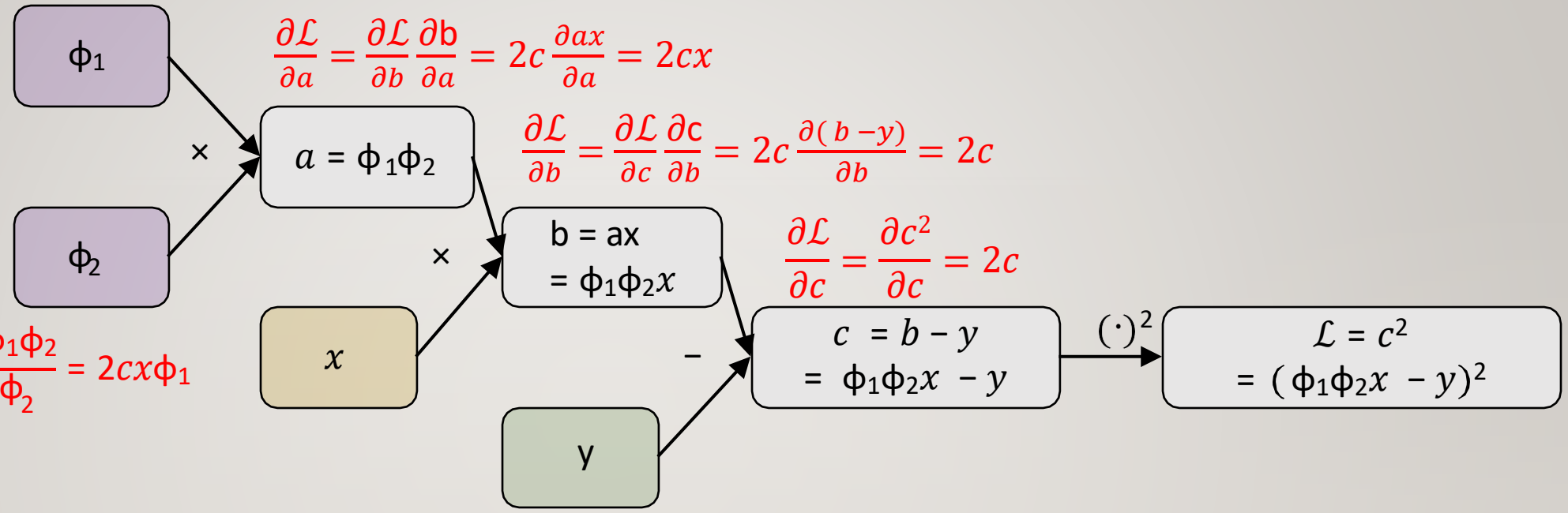


BACKPROPAGATION EXAMPLE

- Let's say:
 - Our dataset has one point, (x, y)
 - Our (weird) model has two parameters, ϕ_1 and ϕ_2 , and predicts $\phi_1\phi_2x$
 - Our loss function will be $\mathcal{L} = (\phi_1\phi_2x - y)^2$
- We build a computational graph with all operations and intermediate variables
 - $a = \phi_1\phi_2$
 - $b = ax = \phi_1\phi_2x$
 - $c = b - y = ax - y = \phi_1\phi_2x - y$
 - $\mathcal{L} = c^2 = (b - y)^2 = (ax - y)^2 = (\phi_1\phi_2x - y)^2$

$$\frac{\partial \mathcal{L}}{\partial \phi_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \phi_1} = 2cx \frac{\partial \phi_1 \phi_2}{\partial \phi_1} = 2cx\phi_2$$

$$\frac{\partial \mathcal{L}}{\partial \phi_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \phi_2} = 2cx \frac{\partial \phi_1 \phi_2}{\partial \phi_2} = 2cx\phi_1$$



Forward step

- The loss $\mathcal{L}$ is computed starting from the “inputs” ϕ_1, ϕ_2, x, y

Backward step (backpropagation)

- The loss $\mathcal{L}$ is used to compute the derivative w.r.t. $c \rightarrow \frac{\partial \mathcal{L}}{\partial c}$
- The derivative $\frac{\partial \mathcal{L}}{\partial c}$ is used to compute the derivative w.r.t. $b \rightarrow \frac{\partial \mathcal{L}}{\partial b}$
- The derivative $\frac{\partial \mathcal{L}}{\partial b}$ is used to compute the derivative w.r.t. $a \rightarrow \frac{\partial \mathcal{L}}{\partial a}$
- The derivative $\frac{\partial \mathcal{L}}{\partial a}$ is used to compute the derivative w.r.t. $\phi_1, \phi_2 \rightarrow \frac{\partial \mathcal{L}}{\partial \phi_1}, \frac{\partial \mathcal{L}}{\partial \phi_2}$

$$\frac{\partial \mathcal{L}}{\partial \phi_1} = 2(\phi_1 \phi_2 x - y)x\phi_2$$

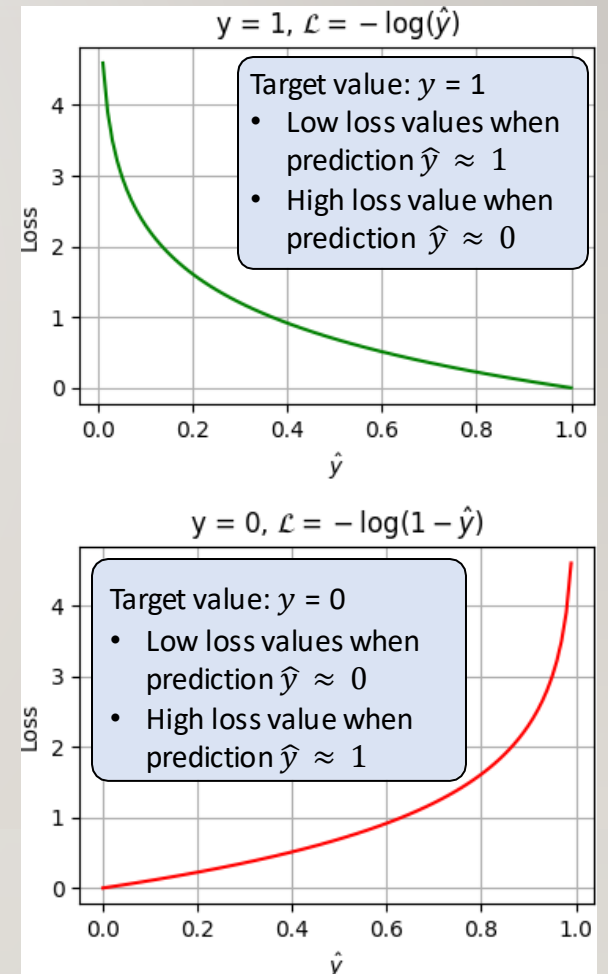
$$\frac{\partial \mathcal{L}}{\partial \phi_2} = 2(\phi_1 \phi_2 x - y)x\phi_1$$

IMPACT ON THE LOSS FUNCTIONS

- Regression
 - *Mean Squared Error, Mean Absolute Error*
- Binary
 - *Binary Cross-Entropy (BCE)*
 - $y = \{0, 1\} \rightarrow$ ground truth
 - $\hat{y} = \text{model } x \in [0,1] \rightarrow$ predicted value

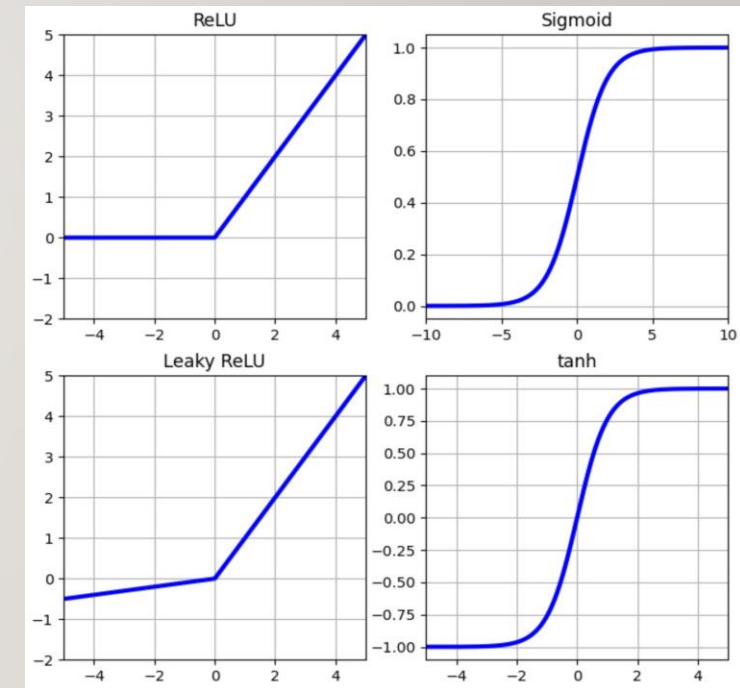
$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- y (ground truth) acts as a “**selector**” of the loss term to be applied
 - $y = 1 \rightarrow \mathcal{L} = -y \log(\hat{y})$
 - $y = 0 \rightarrow \mathcal{L} = -\log(1 - \hat{y})$



THE IMPORTANCE OF THE ACTIVATION FUNCTION

- Not all activation functions can be easily used in Deep Neural Networks
 - Some activation function may suffer the vanishing gradients problem
 - The gradients become too small during backpropagation, causing them to "vanish." As a result, early-layer weights near the input layer receive minimal updates reducing the learning capability
 - E.g., sigmoid or tanh, as the output of is bounded
- ReLU family allows for better gradient flow, promoting more effective training of deep networks



HOW TO ACTUALLY TRAIN THE NETWORK

HOW TO TRAIN IN REALITY?

1. Define the problem and the data
 - Identify the **objective**: e.g., classification, regression, clustering, anomaly detection, generation, etc..
 - Data **preprocessing** the data: cleaning, augmenting, balancing, transformation
 - Split the data: training, validation, test
 - Define the **batch size**
2. **Model engineering** chose the deep neural network **architecture**
3. Select an **optimization algorithm** and the right **Loss function**
4. Train the network and (if available) **validate** and optimize the model doing **hyperparameter tuning**
 1. Evaluate the performance
 2. Study **overfitting** and **underfitting** problems

MINI-BATCH SIZE

- The mini-batch size influence the time for training and the performance of the model
 - Common practice, use mini-batch size as power of 2
 - Improves computationally efficiency
 - **Batch size 1:** is a streaming version of training
 - **PRO:** Fast updates and does not require a lot of memory (only 1 sample)
 - **CONS:** Noisy and slow convergence as we recompute the gradient and update a lot of time
 - **Batch 16-64:** small, used with small dataset and limited hardware
 - **PRO:** good generalization if the dataset is small
 - **CONS:** slow updates
 - **Batch 128-512:** medium – common
 - **PRO:** good trade-off between model performance and training time
 - **Batch 1024+:** large – usable if the dataset is very large
 - **PRO:** faster training and gradient is smoothed
 - **CONS:** may have poor generalization

DIFFERENT PROBLEM DIFFERENT LOSS FUNCTION

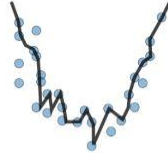
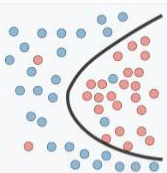
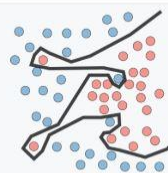
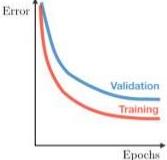
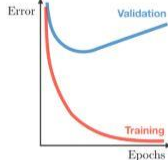
- The loss function $\mathcal{L}$ drives the training process
 - Different goal may use different loss function
- **Regression Loss Functions**
 - **MSE** = $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
 - Simple to compute and differentiable
 - **MAE** = $\frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$
 - Less sensitive to outliers
- **Classification**
 - Binary Cross-Entropy (Log Loss) = $-\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
 - y_i is the actual binary label $\hat{y}_i$ is the probability
 - Differentiable and commonly used in neural network
 - For Categorical Cross-Entropy $-\sum_{i=1}^C y_i \log(\hat{y}_i)$
 - y_i is a vector with the encoded **true label** using **one-hot encoding**
 - $\hat{y}_i$ is the vector with the **predictions** using the **softmax**

PERFORMANCE EVALUATION METRIC

- Loss function is fundamental for the machine to train the network!
 - But having two models the one with the best Loss is not the one we may choose my definition!
- Focus on performance metrics!
- **Classification**
 - How good is the model overall - Accuracy = $\frac{\text{Correct Prediction}}{\text{All Prediction}}$
 - Warning for class imbalance!!
 - How good is the model per Class C_i
 - Precision(C_i) = $\frac{\text{Correct Prediction (Ci)}}{\text{Wrong points belonging to a class Cj Predicted as (C_i) + Correct Prediction (Ci)}}$
 - What proportion of elements identified as C_i are true C_i ?
 - Recall (C_i) = $\frac{\text{Correct Prediction (Ci)}}{\text{Points of Ci Predicted as (C_j) + Correct Prediction (C_i)}}$
 - What proportion of c_i was correctly identified?
 - F1-Score(C_i) = $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
 - Combine the previous two

UNDER AND OVERFITTING

- While training the model we can observe the loss function to identify overfitting and underfitting situations
 - **Underfitting** is when the model is too simple
 - The Loss is high even on training
 - **Overfitting** is when the model is perfect only for training data.
 - The loss function of the training set decreases, getting values even close to 0, while the loss function of the validation set increases

	Just right	Overfitting
Symptoms	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration		
Classification illustration		
Deep learning illustration		
Possible remedies		• Perform regularization • Get more data

[Abhishek Shrivastava, "Underfitting Vs Just right Vs Overfitting in Machine learning", "Keggle"](#)

ADDRESS UNDERFITTING

- **Improve preprocessing:**
 - removing noise from the data, evaluate the presence of outliers, change of the categorical features are encoded, etc.
- **Improve model engineering:**
 - Increase model complexity (e.g., increasing the number of neurons), change the architecture, Increase the number of epochs, change the batch size

ADDRESS OVERFITTING

- Regularization reduce **overfitting** by improving generalization
 - It is crucial in deep learning as neural networks have a large capacity to memorize training data
 - The techniques add constraints or penalties to the learning process, encouraging the model to generalize better

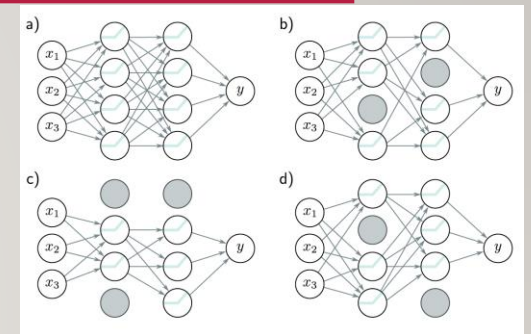
Work on the weights

- **L1 Regularization (Lasso):** Adds to the loss function a penalty proportional to the absolute value of the weights
 - Encourages sparsity in the model by forcing some weights to become zero, effectively selecting features.
 - Penalty term: $\lambda \sum_{i=0}^n |\phi_i|$
 - λ : regularization strength
- **L2 Regularization (Ridge):** Adds to the loss function a penalty proportional to the square of the weights
 - Discourages large weights and spreads the importance across multiple features.
 - Penalty term: $\lambda \sum_{i=0}^n \phi_i^2$

ADDRESS OVERFITTING

Work on the architecture

- **Dropout:** randomly "drops" (sets to zero the output of) a subset of neurons during each **training iteration**
 - Gray nodes
 - Force the network to learn more robust features by preventing neurons from becoming overly dependent on each other
 - During prediction all neurons are used



Work on training process

- **Early Stopping:** stops the training process when the performance on a validation set starts to degrade. This prevents the model from continuing to fit the training data too closely



Work on the data

- **Batch Normalization:** Normalizes the inputs of one (or more) layer(s) to stabilize and speed up training.
 - It can reduce overfitting by making the training process less sensitive to weight initialization
 - Can be applied also **before** the activation function
- **Noise Injection:** adds random noise to inputs, weights, or activations during training to make the model more robust

